



Bit-precise integers

Document number:

[P3666R0](#)

Date:

2025-09-11

Audience:

SG6, SG22, EWG, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3666/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/bitint.cow



ABSTRACT: C23 has introduced so-called "bit-precise integers" into the language, which should be brought to C++ for compatibility, among other reasons. Following an exploration of possible designs in [\[P3639R0\]](#) "The `_BitInt` Debate", this proposal introduces a new set of fundamental types to C++.

§ Contents

- 1 **Revision history**
- 2 **Introduction**
 - 2.1 C23
 - 2.2 P3140R0 "`std::int_least128_t`"
 - 2.3 P3639R0 "The `_BitInt` Debate"
- 3 **Motivation**
 - 3.1 Computation beyond 64 bits
 - 3.2 Cornerstone of standard library facilities
 - 3.3 C ABI compatibility
 - 3.4 Resolving issues with the current integer type system
 - 3.5 Portable exact-width integers
- 4 **Core design**



- 4.1 Why not a class template?
 - 4.1.1 Full C compatibility requires fundamental types
 - 4.1.2 Common spelling of `unsigned _BitInt(N)`
 - 4.1.3 C compatibility would require an enormous amount of operator overloads etc.
 - 4.1.4 Constructors cannot signal narrowing
 - 4.1.5 Tiny integers are useful in C++
 - 4.1.6 Special deduction rules
 - 4.1.7 Special overload resolution rankings
 - 4.1.8 Quality of implementation requires a fundamental type
- 4.2 Why the `_BitInt` keyword spelling?
- 4.3 Underlying type of enumerations
- 4.4 Should bit-precise integers be optional?
- 4.5 `_BitInt(1)`
- 4.6 Undefined behavior on signed integer overflow
- 4.7 Permissive implicit conversions
 - 4.7.1 C compatibility
 - 4.7.2 Difficulty of carving out exceptions in the language
 - 4.7.3 Picking some low-hanging fruits
 - 4.7.4 Conclusion on implicit conversions
- 4.8 Raising the `BITINT_MAXWIDTH`
 - 4.8.1 Possible increased `BITINT_MAXWIDTH` values
- 4.9 Template argument deduction
- 4.10 No preprocessor changes, for better or worse
- 5 **Library design**
 - 5.1 Naming of the alias template
 - 5.1.1 Why no `_t` suffix?
 - 5.2 `format`, `to_chars`, and `to_string` support for bit-precise integers
 - 5.3 Preventing `ranges::iota_view` ABI break
 - 5.4 Bit-precise `size_t`, `ptrdiff_t`
 - 5.5 New `abs` overload
 - 5.6 Using bit-precise integers in `<cmath>` functions
 - 5.7 Lack of random number generation support
 - 5.8 `simd` support for bit-precise integers
 - 5.8.1 `simd` design problems
 - 5.8.2 `simd` design conclusion
 - 5.9 `valarray` support for bit-precise integers
 - 5.10 Broadening `is_integral`
 - 5.11 Miscellaneous library support
 - 5.12 Feature testing
 - 5.13 Passing `bit_int` into standard library function templates
- 6 **Education**
 - 6.1 Teaching principles



7	Implementation experience
8	Impact on the standard
8.1	Impact on the core language
8.2	Impact on the standard library
9	Wording
9.1	Core
9.1.1	[lex.icon]
9.1.2	[basic.fundamental]
9.1.3	[conv.rank]
9.1.4	[conv.prom]
9.1.5	[dcl.type.general]
9.1.6	[dcl.type.simple]
9.1.7	[dcl.enum]
9.1.8	[temp.deduct.general]
9.1.9	[temp.deduct.type]
9.1.10	[cpp.predefined]
9.2	Library
9.2.1	[version.syn]
9.2.2	[cstdint.syn]
9.2.3	[climits.syn]
9.2.4	[stdbit.h.syn]
9.2.5	[range.iota.view]
9.2.6	[alg.foreach]
9.2.7	[alg.search]
9.2.8	[alg.copy]
9.2.9	[alg.fill]
9.2.10	[alg.generate]
9.2.11	[charconv.syn]
9.2.12	[charconv.to.chars]
9.2.13	[charconv.from.chars]
9.2.14	[string.syn]
9.2.15	[string.conversions]
9.2.16	[cmath.syn]
9.2.17	[c.math.abs]
9.2.18	[simd.general]
9.2.19	[numerics.c.ckdint]
9.2.20	[atomics.ref.int]
9.2.21	[atomics.types.int]
10	Acknowledgements
11	References

§ 1. Revision history



This is the first revision.

§ 2. Introduction

In distant history, there have been various attempts at standardizing multi-precision integers in C++, such as [\[N1692\]](#) "A Proposal to add the Infinite Precision Integer to the C++ Standard Library", [\[N1744\]](#) "Big Integer Library Proposal for C++0x", and [\[N4038\]](#) "Proposal for Unbounded-Precision Integer Types", all of which have been abandoned by the authors. However, there has always been some enthusiasm in the committee for such a feature.

I am picking up where they have left off. Whether this results in a C++ feature or adds corpses to the graveyard of multi-precision papers remains to be seen.

§ 2.1. C23

Recently, WG14's [\[N2763\]](#) introduced the `_BitInt` set of types to the C23 standard, and [\[N2775\]](#) further enhanced this feature with literal suffixes. For example, this feature may be used as follows:

```
// 8-bit unsigned integer initialized with value 255.  
// The literal suffix wb is unnecessary in this case.  
unsigned _BitInt(8) x = 0xFFwb;
```

In short, the behavior of these *bit-precise integers* is as follows:

- No integer promotion to `int` takes place.
- Mixed-signedness comparisons, implicit conversions, and other permissive feature are supported.
- They have lower conversion rank than standard integers, so an operation between `_BitInt(8)` and `int` yields `int`, as does an operation with `_BitInt(N)` where N is the width of `int`. They only have greater conversion rank when their width is greater.
- Widths up to `BITINT_MAXWIDTH` are allowed, with padding bits being added if needed. `BITINT_MAXWIDTH` is at least 64.

§ 2.2. P3140R0 "`std::int_least128_t`"

In parallel, I proposed [\[P3140R0\]](#) which would add 128-bit integers as `std::int_least128_t` to the C++ standard. It became apparent to me that standardizing just a single width of 128 and not solving the `_BitInt` C compatibility problem would be futile, so I've stepped away from the proposal. However, the feedback and experience gained from P3140 made it well worth the time spent.

§ 2.3. P3639R0 "The `_BitInt` Debate"



I've subsequently proposed [P3639R0] "The `_BitInt` Debate", which shifts the goal to compatibility with C's `_BitInt` type, and attempts to answer whether the set of types corresponding to `_BitInt` should be a class template or a family of fundamental types. P3639R0 received much feedback in 2025. First, from SG22:

”

The WG14 delegation to SG22 believes that the C++ type family that deliberately corresponds to `_BitInt` (perhaps via compatibility macros) should be... (Fundamental/Library)

SF	F	N	L	SL
8	1	1	0	0

WG21

SF	F	N	L	SL
4	5	0	0	0

The overall sentiment in SG22 was that a fundamental type is "inevitable". This is reflected in the polls. SG6 also saw the paper, but had no clear opinion on the fundamental/library problem. Last but not least, EWG also saw the paper in Sofia 2025, with the following two polls:

”

P3639R0: EWG prefers that `_BitInt`-like type be a FUNDAMENTAL TYPE (in some form) in C++.

SF	F	N	A	SA
13	9	9	5	4

Result: consensus

P3639R0: EWG prefers that `_BitInt`-like type be a LIBRARY TYPE (in some form) in C++.

SF	F	N	A	SA
8	9	14	8	3

Result: not consensus

While the general direction for the paper is clear (fundamental type), there are many contentious design issues, such as the minimum supported `BITINT_WIDTH` or how permissive bit-precise integers should be with implicit conversions. You can identify the most contentious problems as blocks such as:



DECISION: This is an example.

§ 3. Motivation

§ 3.1. Computation beyond 64 bits



Computation beyond 64-bit bits, such as with 128-bits is immensely useful. A large amount of motivation for 128-bit computation can be found in [P3140R0]. Computations in cryptography, such as for RSA require even 4096-bit integers.

Even when performing most operations using 64-bit integers, there are certain use cases where temporarily, twice the width is needed. For example, the implementation of `linear_congruential_engine<uint64_t>` requires the user of 128-bit arithmetic, as does arithmetic with 64-bit [fixed-point numbers](#) (e.g. Q32.32).

§ 3.2. Cornerstone of standard library facilities

There are various existing and possible feature library facilities that would greatly benefit from an N-bit integer type:

- As mentioned above, the implementation of `linear_congruential_engine<uint64_t>` requires the use of 128-bit integers.
- `bitset` has constructors taking `unsigned long long` and a `to_ullong` member function that converts from/to integers. This is clunky and limited considering that bitsets can be much larger than `unsigned long long`. Bit-precise integers would be a superior alternative to `unsigned long long` here.
- [P3161R4] proposes library features such as `add_carry` or `mul_wide` which produce a wider integer result than the operands. For example:

```
template<class T>
struct mul_wide_result {
    T low_bits;
    T high_bits;
};
template<class T>
constexpr mul_wide_result<T> mul_wide(T x, T y) noexcept;
```

Proposals like these are arguably obsolete if the same operation can be expressed by simply casting the operands to an integer with double the width prior to the multiplication.

§ 3.3. C ABI compatibility

C++ currently has no portable way to call C functions such as:

```
_BitInt(32) plus(_BitInt(32) x, _BitInt(32) y);
_BitInt(128) plus(_BitInt(128) x, _BitInt(128) y);
```

While one could rely on the ABI of `uint32_t` and `_BitInt(32)` to be identical in the first overload, there certainly is no way to portably invoke the second overload.

This compatibility problem is not a hypothetical concern either; it is an urgent problem. There are already targets with `_BitInt` supported by major compilers, and used by C developers: 

Compiler	<code>BITINT_MAXWIDTH</code>	Targets	Languages
clang 16+	8'388'608	all	C & C++
GCC 14+	65'535	64-bit only	C
MSVC	✗	✗	✗

§ 3.4. Resolving issues with the current integer type system

`_BitInt` as standardized in C solves multiple issues that the standard integers (`int` etc.) have. Among other problems, integer promotion can result in unexpected signedness changes.



EXAMPLE: The following code has undefined behavior if `int` is a 32-bit signed integer (which it is on many platforms).

```
uint16_t x = 65'535;  
uint16_t y = x * x;
```

During the multiplication `x * x`, `x` is promoted to `int`, and the result of the multiplication `4'294'836'225` is not representable as a 32-bit signed integer. Therefore, signed integer overflow takes places ... given unsigned operands.



EXAMPLE: The following code may have surprising effects if `std::uint8_t` is an alias for `unsigned char` and gets promoted to `int`.

```
std::uint8_t x = 0b1111'0000;  
std::uint8_t y = ~x >> 1; // y = 0b1000'01111
```

Surprisingly, `y` is not `0b111` because `x` is promoted to `int` in `~x`, so the subsequent right-shift by `1` shifts one set bit into `y` from the left. Even more surprisingly, if we had used `auto` instead of `std::uint8_t` for `y`, `y` would be `-121`, despite our code seemingly using only unsigned integers.

Overall, the current integer promotion semantics are extremely surprising and make it hard to write correct code involving promotable unsigned integers. Promotion also makes it hard to expose small integers (e.g. 10-bit unsigned integer) that exist in hardware (e.g. FPGA) in the language, since all operations would be performed using `int`. Unconventional hardware such as FPGAs are pillar of the motivation for `_BitInt` laid out in [N2763].

§ 3.5. Portable exact-width integers

There is no portable way to use an integer with exactly 32 bits in standard C++. `int_least32_t` and `long` may be wider, and `int32_t` is an optional type alias which only exists if such an integer type has no padding bits. Having additional non-padding bits may be undesirable when

implementing serialization, networking, etc. where the underlying file format or network protocol is specified using exact widths.



While most platforms support 32-bit integers as `int32_t`, their optionality is a problem for use in the standard library and other ultra-portable libraries. There are many use cases where padding bits would be an acceptable sacrifice in exchange for writing portable code, and bit-precise integers fill that gap in the language.

§ 4. Core design

The overall design strategy is as follows:

- The proposal is a C compatibility proposal first and foremost. Whenever possible, we match the behavior of the C type.
- The goal is to deliver a minimal viable product (MVP) which can be integrated into the standard as quickly as possible. This gives us plenty of time to add standard library support wherever desirable over time, as well as other convenience features surrounding `_BitInt`.

§ 4.1. Why not a class template?

[P3639R0] explored in detail whether to make it a fundamental type or a library type. Furthermore, feedback given by SG22 and EWG was to make it a fundamental type, not a library type. This boils down to two plausible designs (assuming `_BitInt` is already supported by the compiler), shown below.

F – Fundamental type	L – Library type
<pre>template <size_t N> using bit_int = _BitInt(N); template <size_t N> using bit_uint = unsigned _BitInt(N);</pre>	<pre>template <size_t N> class bit_int { private: _BitInt(N) _M_value; public: // ... }; template <size_t N> class bit_uint { /* ... */; };</pre>

The reasons why we should prefer the left side are described in the following subsections.

§ 4.1.1. Full C compatibility requires fundamental types

`_BitInt` in C can be used as the type of a bit-field, among other places:

```
struct S {
    // 1. _BitInt as the underlying type of a bit-field
```




```

    _BitInt(32) x : 10;
};

// 2. _BitInt in a switch statement
_BitInt(32) x = 10;
switch (x) {}

// 3. _BitInt used as a null pointer constant
void* p = 0wb;

// 4. _BitInt used as underlying type of enumeration
//     (NOT valid now, but may be in the future)
enum S : _BitInt(32) { X = 0 };

```

Since C++ does not support the use of class types in bit-fields, such a `struct S` could not be passed from C++ to a C API. A developer would face severe difficulties when porting C code which makes use of these capabilities to C++ and if bit-precise integers were a class type in C++.

§ 4.1.2. Common spelling of `unsigned _BitInt(N)`

If bit-precise integers were class types in C++, this would cause a serious problem with a common spelling that can be used in both C and C++ headers, even if there was a `_BitInt` compatibility macro.

```

#define _BitInt(...) std::bit_int<__VA_ARGS__>

unsigned _BitInt(8) x; // error: cannot combine 'unsigned' with class type

```

There are some workarounds to the problem, but they all seem unattractive:

- Permitting `signed` and `unsigned` to be combined with class types in general, perhaps with the effect of applying `std::make_signed` and `std::make_unsigned`. This would lead to a bifurcation of the language where both a builtin feature and a type trait achieve the same effect.
- "Blessing" the `std::bit_int<N>` *type-name* so it can be combined with `unsigned`. This would be a highly unusual special case in the language.
- Making `_BitInt(...)` expand to an unspecified construct that can be combined with `signed` and `unsigned`. This means there needs to be a fundamental type, although that fundamental type only acts as a proxy for the `std::bit_int` class type. Once again, this comes off as an unusual special case.
- Introducing a `_BitUint(...)` macro for unsigned bit-precise integers, and insisting that both C and C++ developers use this for interoperability. This feels like an unnecessary burden for C developers considering that their spelling works perfectly fine and that we have other design options which keep C code intact.

§ 4.1.3. C compatibility would require an enormous amount of operator overloads etc.

Integer types can be used in a large number of places within the language. If we wanted a `std::bit_int` class type to be used in the same places (which would be beneficial for C-

interoperable code), we would have to add a significant amount of operator overloads and user-defined conversion functions:



- There are conversion to/from floating-point types and other integral types.
- There are conversion to/from enumeration types.
- There are conversion to/from pointers, at least for `_BitInts` of the same width as `uintptr_t`.
- Integers can be used to add offsets onto pointers, and by proxy, in the subscript operator of builtin arrays.
- Arithmetic operators can be used to operate between any mixture of arithmetic types, such as `_BitInt(32) + float`.

Any discrepancies would lead to some code using bit-precise integers behaving differently in C and C++, which is undesirable.

Furthermore, the *wb integer-suffix* for `_BitInt` is fairly complicated to implement as a library feature because the resulting type depends on the numeric value of the literal. This means it would presumably be implemented like:

```
template<char... Chars>
constexpr auto operator""wb() { /* ... */ }
template<char... Chars>
constexpr auto operator""WB() { /* ... */ }
template<char... Chars>
constexpr auto operator""uwb() { /* ... */ }
template<char... Chars>
constexpr auto operator""UWB() { /* ... */ }
template<char... Chars>
constexpr auto operator""uWB() { /* ... */ }
template<char... Chars>
constexpr auto operator""Uwb() { /* ... */ }
```

Seeing that properly emulating C's behavior for `_BitInt` (and its suffixes) requires a mountain of complicated operator overload sets, user-defined conversion functions, converting constructors, and user-defined literals, it seems unreasonable to go this direction.

A major selling point of a library type is that library types have more teachable interfaces, since the user simply needs to look at the declared members of the class to understand how it works. If the interface is a record-breaking convoluted mess, this benefit is lost. If we choose not to add all this functionality, then we lose a large portion of C compatibility. Either option is bad, and making `std::bit_int` a fundamental type seems like the only way out.

§ 4.1.4. Constructors cannot signal narrowing

Some C++ users prefer list initialization because it prevents narrowing conversion. This can prevent some mistakes/questionable code:

```
unsigned x = -1; // OK, x = UINT_MAX, but this looks weird
unsigned y{ -1 }; // error: narrowing conversion
```

This would not be feasible if `std::bit_int` was a library type because narrowing cannot be signaled by constructors. Consider that `std::bit_int` and `std::bit_uint` should have a non-explicit constructor (template) accepting `int` (and other integral types) to enable compatibility in situations like:

```
#ifdef __cplusplus
typedef std::bit_uint<32> u32; // C++
#else
typedef unsigned _BitInt(32) u32; // C
#endif
// Common C and C++ code, possibly in a header:

// OK, converting int → u32.
// Using "incorrectly typed" zeros is fairly common, both in C and in C++.
u32 x = 0;
// OK, same conversion, but would be considered narrowing in C++.
// Not very likely to be written.
u32 y = -1;
```

If such a `std::bit_uint<32>(int)` constructor existed, the following C++ code would not raise any errors:

```
std::bit_uint<32> x{ 0 }; // OK, as expected
std::bit_uint<32> y{ -1 }; // OK?! But this looks narrowing!
```

This code simply calls a `std::bit_uint<32>(int)` constructor, and while the initialization of `y` is *spiritually* narrowing, no narrowing conversion actually takes place. In conclusion, if `std::bit_int` was a library type, C++ users who use this style would lose what they consider a valuable safety guarantee.

NOTE: It can be argued that using list-initialization for this purpose is an anti-pattern and only solves a subset of the issues that compiler warnings and linter warnings should address. Personally, I have no strong position on this issue.

§ 4.1.5. Tiny integers are useful in C++

In some cases, tiny `bit_int`'s may be useful as the underlying type of an enumeration:

```
enum struct Direction : bit_int<2> {
    north, east, south, west,
};
```

By using `bit_int<2>` rather than `unsigned char`, every possible value has an enumerator. If we used e.g. `unsigned char` instead, there would be 252 other possible values that simply have no name, and this may be detrimental to compiler optimization of `switch` statements etc.

NOTE: Using bit-precise integers as the underlying type of enumerations is not currently proposed (§4.3. Underlying type of enumerations). However, it is likely that it

will be supported in the future (after some follow-up proposal), so I still consider this motivation to be valid.



§ 4.1.6. Special deduction rules

While this proposal focuses on the minimal viable product (MVP), a possible future extension would be new deduction rules allowing the following code:

```
template <size_t N>
    void f(bit_int<N> x);

f(int32_t(0)); // calls f<32>
```

Being able to make such a call to `f` is immensely useful because it would allow for defining a single function template which may be called with every possible signed integer type, while only producing a single template instantiation for `int`, `long`, and `_BitInt(32)`, as long as those three have the same width. The prospect of being able to write bit manipulation utilities that simply accept `bit_uint<N>` is quite appealing.

If `bit_int<N>` was a class type, this would not work because template argument deduction would fail, even if there existed an implicit conversion sequence from `int32_t` to `bit_int<32>`. These kinds of deduction rules may be shutting the door on this mechanism forever.

§ 4.1.7. Special overload resolution rankings

Yet another possible future extension would be rankings for overload resolution that take integer width into account.



EXAMPLE: Special overload rankings could make bit-precise integers more easily interoperate with existing overload sets:

```
struct QString { // see Qt 6 documentation
    static QString number(int n, int base = 10);
    static QString number(long n, int base = 10);
    static QString number(long long n, int base = 10);
    // ...
};

QString::number(0wb); // currently ambiguous, but could call QString::nu
```

This could be valid if `number(int)` was considered a better match on the basis that its width is closer to that of `0wb`. Further disambiguation could be applied if `int` and `long` had the same width.



EXAMPLE: Special overload rankings could make it possible to create non-template overload sets that cover a greater range of widths:

```
bit_uint<64>  clmul_wide(bit_uint<32>);
bit_uint<128> clmul_wide(bit_uint<64>);

clmul_wide(128wb); // OK, calls clmul_wide(bit_uint<32>)
```



These overload ranking rules would be difficult or impossible to define using a class type. Of course, they are not proposed, and it's not certain whether such rules are desirable to have, but it would be unfortunate to shut the door on these possible features forever.

§ 4.1.8. Quality of implementation requires a fundamental type

While a library type `class bit_int` gives the implementation the option to provide no builtin support for bit-precise integers, to achieve high-quality codegen, a fundamental type is *inevitably* needed anyway.



EXAMPLE: When an integer division has a constant divisor, like `x / 10`, it can be optimized to a fixed-point multiplication, which is much cheaper:

```
unsigned div10(unsigned x) {
    return x / 10;
}
```

For this operation, Clang emits the following assembly:

```
div10(unsigned int):
    mov     ecx, edi
    mov     eax, 3435973837
    imul    rax, rcx
    shr     rax, 35
    ret
```

Basically, the result is rewritten as `x * 3435973837ull >> 35`. This optimization is called *strength reduction* and may lead to dramatically faster code, especially when the hardware has no direct support for integer division. Similarly, multiplication can be strength-reduced to bit-shifting when a factor is a power of two, remainder operations can be reduced to bitwise AND when the divisor is a power of two, etc.

Performing strength reduction requires the compiler to be aware that a division is taking place, and this fact is lost when division is implemented in software, as a loop which expands to hundreds of IR instructions when unrolled.

Furthermore, the compiler frontend needs to understand certain operations to warn about obvious mistakes such as division by zero, shifting by an overly large amount, producing signed integer overflow unconditionally, etc. Use of `pre` on e.g. `bit_int::operator/` cannot be used to achieve this because numerics code needs to have no hardened preconditions and no contracts, for performance reasons.

Last but not least, a fundamental type is needed to speed up constant evaluation. Something like integer division between two `bit_int<128>` may be much faster as a compiler-builtin

operation compared to constant-evaluating a "software division" loop with 128 iterations necessary to implement [binary division](#).



If we accept the premise that a fundamental type is needed anyway (possibly as an implementation detail of a class template), then the class template actively harmful bloat:

- Any arithmetic operation needs to go through overload resolution, competing with countless other [operator](#)+s (there are many in the standard library already). Even if implementers special-case these operations to circumvent the (usually awful) diagnostic quality of a failed call to [operator](#)+, there remains substantial cost: overload resolution is expensive.
- Every distinct `bit_int<N>` and `bit_uint<N>` would be a separate instantiation of a relatively large class template, which would undoubtedly add compilation cost.
- Invocations of member functions or operator overloads may add cost to debug builds and constant evaluation.

§ 4.2. Why the `_BitInt` keyword spelling?

I also propose to standardize the keyword spelling `_BitInt` and `unsigned _BitInt`. I consider this to a "C compatibility spelling" rather than the preferred one which is taught to C++ developers. See also [§6.1. Teaching principles](#).

While a similar approach could be taken as with the `_Atomic` compatibility macro, macros cannot be exported from modules, and macros needlessly complicate the problem compared to a keyword. Furthermore, to enable compiling shared C and C++ headers, all of the spellings `_BitInt`, `signed _BitInt` and `unsigned _BitInt` need to be valid. This goes far beyond the capabilities that a compatibility macro like `_Atomic` can provide without language support. If the `_BitInt(...)` macro simply expanded to `bit_int<__VA_ARGS__>`, this may result in the ill-formed code `signed bit_int<N>`.

The most plausible fix would be to create an exposition-only *bit-int* spelling to enable `signed bit-int<N>`, which makes our users beg the question:

”

Why is there a compatibility macro for an exposition-only keyword spelling?! Why are we making everything more complicated by not just copying the keyword from C?! Why is this exposition-only when it's clearly useful for users to spell?!

The objections to a keyword spelling are that it's not *really* necessary, or that it "bifurcates" the language by having two spellings for the same thing, or that those ugly C keywords should not exist in C++. Ultimately, it's not the job of WG21 to police code style; both spellings have a right to exist:

- The `bit_int` alias template fits in aesthetically with the rest of the language, and conveys clearly (via "pointy brackets") that the given width is a constant expression.
- The `_BitInt` spelling is useful for writing C/C++-interoperable code, and C compatibility is an important design goal.

It seems like both spellings are going to exist, whether `_BitInt` is a keyword or compatibility macro. Since there is no clear technical benefit to a macro, the keyword is the *only* logical choice.



NOTE: Clang already supports the `_BitInt` keyword spelling as a compiler extensions, so this is standardizing existing practice.

§ 4.3. Underlying type of enumerations

The following C code is ill-formed:

```
// error: '_BitInt(32)' is an invalid underlying type  
enum S : _BitInt(32) { x = 0 };
```

There is no obvious reason why `_BitInt` must not be a valid underlying type. However, since this feature is not needed for C compatibility, I do not consider bit-precise integers as underlying types to be part of the MVP. A follow-up proposal could add this feature later, possibly in coordination with WG14.



NOTE: See [N3550] §6.7.3.3 "Enumeration specifiers" for restrictions.

This behavior should be mirrored.

§ 4.4. Should bit-precise integers be optional?

As in C, `_BitInt(N)` is only required to support N of at least `LLONG_WIDTH`, which has a minimum of 64. This makes `_BitInt` a semi-optional feature, and it is reasonable to mandate its existence, even in freestanding platforms.

Of course, this has the catch that `_BitInt` may be completely useless for tasks like 128-bit computation. As unfortunate as that is, the MVP should include no more than C actually mandates. Mandating a greater minimum width could be done in a future proposal.

§ 4.5. `_BitInt(1)`

C23 does not permit `_BitInt(1)` but does permit `unsigned _BitInt(1)`, mostly for historical reasons (C did not always requires two's complement representation for signed integers). This is an irregularity that could make generic programming harder in C++.

However, there are already plans to lift the restriction for C2y; see [N3644]. Following v3 of the proposal, `_BitInt(1)` is expected to be valid type, and `0wb` should be of type `_BitInt(1)` rather than `_BitInt(2)`. That proposal also contains some practical motivation for why a single-bit should be permitted.



NOTE: If `_BitInt(1)` was allowed, it would be able to represent the values 0 and -1, just like an `int x : 1`; bit-field.

§ 4.6. Undefined behavior on signed integer overflow



DECISION: Whether (and how) to address signed integer overflow in bit-precise integers specifically is a contentious issue, which has been discussed in great length. EWG should decide whether to perpetuate undefined behavior or to have different behavior for bit-precise integers.

I propose to perpetuate bit-precise integers having undefined behavior on signed integer overflow, just like `int`, `long` etc. This has a few reasons:

- bit-precise integers have undefined overflow in C, so this is what users are used to.
- "Solving" signed integer overflow for bit-precise integers is not part of the MVP. Undefined behavior can always be defined to do something else, so there is no urgent need for this paper to address this issue, rather than solving it in a follow-up paper.
- Signed integer overflow having undefined behavior is a much broader issue that should be looked at in general, for all integer types, not just bit-precise integer types. Perhaps hardened implementations could have wrapping overflow with erroneous behavior. In any case, the problem exceeds the scope of the paper.
- It is highly unusual that users would expect signed integer overflow to be well-behaved, such as having wrapping behavior. Adding two positive numbers and obtaining a negative number is not typically useful.
- The undefined behavior here is useful. It allows for optimizations such as converting `x + 3 < 0` into `x < -3`.

That being said, much of the feedback surrounding bit-precise integers revolved around signed integer overflow. If we were to make signed integer overflow *not* undefined for bit-precise integers, there are two options that may find consensus:

- Make signed integer overflow wrapping. In other words, most operations would be performed as if by casting to the corresponding unsigned type, performing the operation, and casting back.
- Make signed integer overflow wrapping *and* erroneous. This is mirroring Rust's behavior, and would typically be implemented by detecting overflow on debug builds and in constant evaluation, but ignoring it and letting it wrap in release builds.

§ 4.7. Permissive implicit conversions

Just like any other integral type, the proposal makes bit-precise integers quite permissive when it comes to implicit conversions. This is disappointing to anyone who wants bit-precise integers to be a much "stricter" or "safer" alternative to standard integers, but it is arguably the better design for various reasons.



DECISION: This is a contentious issue, and feedback was given that implicit conversions should be limited for bit-precise integers. The idea of limiting implicit conversions was also a selling point of [\[P3639R0\]](#).

There are several plausible directions:

- Perpetuate implicit conversions of the standard integers. This is the current design of the proposal; see rationale below.
- Limit implicit conversions for bit-precise integers. For example, converting `int` to `unsigned _BitInt(1)` could be invalid.
- Perpetuate implicit conversions of standard integers, but also add a library wrapper or another `_BitIntStrict` type family with much more restrictive semantics.



§ 4.7.1. C compatibility

Firstly, the point of perpetuating implicit conversions is to mirror the C semantics as closely as possible, which leads to few or no surprises when porting code between the languages, or when writing C-interoperable headers.

If we look at how C users use `_BitInt`, [GitHub code search for "_BitInt" language:C](#) yields examples such as:

”

```
// mixing signed and unsigned bit-precise integers
unsigned _BitInt(128) max128s = 0x7FFF'FFFF'FFFF'FFFF'FFFF'FFFF'FFFF'FFFF
// mixing bit-precise and standard integers
unsigned _BitInt(4) a = 1u;
// mixing bit-precise and standard integers of different signedness
unsigned _BitInt(total) bit = 1;
// ... including cases where initialization does not preserve values
unsigned _BitInt(3) max3u = -1;
```

If we were to make implicit conversions much more restrictive on the C++ side, it would become very easy to slip up and accidentally write a header that does not also compile in C++.

§ 4.7.2. Difficulty of carving out exceptions in the language

Writing C++ code involving bit-precise integers would be quite annoying and "flag" many harmless cases if the rules were too strict.



EXAMPLE: The following line of code would not compile if converting from `int` to `bit_uint<8>` was unconditionally ill-formed.

```
std::bit_uint<8> x = 0; // error?
```

`0` is "incorrectly signed" for `std::bit_uint`, and the conversion from `int` to `bit_uint<8>` is not value-preserving generally, but writing code like this is perfectly reasonable.

The workaround would be to use correct literals, such as:

```
std::bit_uint<8> x = 0uwb; // OK, conversion bit_uint<1> → bit_uint<8>
```



To combat this problem, it would be necessary to carve out various special cases. For example, permitting value-preserving conversions with constant expressions would prevent the example above from being flagged.



NOTE: There is precedent for such special casing of value-preserving conversions. Specifically, see mentions of "narrowing" in [\[dcl.init.list\]](#), [\[expr.spaceship\]](#), and [\[expr.const\]](#).

However, such special cases are insufficient to cover *all* harmless cases.



EXAMPLE:

```
void for_each_cell(vec3 x) {  
    for (int i = 0; i < 3; ++i) {  
        do_something(x[i]);  
    }  
}
```

Even though `i` is not a constant expression, `x[i]` will "just work" no matter what integer type `vec3::operator[]` accepts.

Existing C++ code bases that have not used flags such as `-Wconversion` from the start are likely filled with many such harmless cases of mixed-sign implicit conversions. If bit-precise integer types were introduced into these code bases, refactoring effort may be unacceptable.

Furthermore, discrepancies between the standard integers and bit-precise integers would make it much harder to write generic code:



EXAMPLE: The following function template may be instantiated with any integral type `T`, but the instantiation would be ill-formed for `T = unsigned _BitInt(8)` with restrictive implicit conversions:

```
template<std::integral T>  
T div_ceil(T x, T y) { // performs integer division, rounding to +inf  
    // ⚠ Could be mixed-sign comparison:  
    bool quotient_positive = (x ^ y) >= 0;  
    // ⚠ Could be mixed-sign comparison  
    bool adjust = x % y != 0 && quotient_positive;  
    // ⚠ Could be mixed-sign addition between int (0 or 1)  
    // and unsigned _BitInt(N) "x / y":  
    // ⚠ Could be lossy conversion when returning: int → unsigned _BitInt  
    return x / y + int(adjust);  
}
```

Literally every statement of this template may fail to compile when `T = unsigned\_BitInt\(8\)`, depending on how strict implicit conversions are. I conjecture that there are vast amounts of templates like `div_ceil`. To accommodate bit-precise integers in this function, a rewrite is necessary:

```
template<std::integral T>
T div_ceil(T x, T y) {
    constexpr auto zero = T(0);
    bool quotient_positive = (x ^ y) >= 0 ? zero;
    bool adjust = x % y != 0 ? zero && quotient_positive;
    return x / y + int T(adjust);
}
```



EXAMPLE: The following function template involves a mixed-sign operation, but is entirely harmless for any type `T`:

```
constexpr unsigned mask = 0xf;
T integer = /* ... */;
x &= mask; // equivalent to x = x & mask;
```

Even if `x` is signed instead of unsigned, `x & mask` produces a mathematically identical result.

§ 4.7.3. Picking some low-hanging fruits

While conversions between bit-precise integers and other signed or unsigned integer could be difficult to restrict due to the reasons above, other conversions are much more rare and could more easily be restricted:

- Conversions between bit-precise integers and `bool`.
- Conversions between bit-precise integers and character types.
- Conversions between bit-precise integers and floating-point types.

It would be reasonable to ban these conversions unconditionally because they are likely to be category errors.



BUG: Consider the "easter egg" discovered in cplusplus.com/forum/general/105627/:

” I was fixing a couple of minor bugs in a program I've been working on, when I made the mistake of typing `cout<<string('\n', 1);` instead of `cout<<string(1, '\n');`

I didn't get any compile errors and the programs reaction gave me a bit of a laugh. Instead of the blank line I wanted to put in, I got `:):):):):):):):):):):)` (10 of them). It just made me wonder as a relative C++ beginner what other "easter eggs" are there that people might feel like sharing.

It turns out that `string('\\n', 1)` is not an "easter egg", it just results in the Windows terminal displaying a `char(1)` as ":" ten times. The `string(size_t, char)` overload is called, and since the '\\n' and 1 can be converted to `size_t` and `char` without any change in value, compilers generally don't raise a warning, even with `-Wconversion` enabled.

The least harmful of these conversions is a value-preserving conversion from a bit-precise integer to a floating-point type. However, at best, these lack clarity of intent.



EXAMPLE: Consider a code base with the following two functions computing $\sqrt{x}$:

```
int isqrt(int x);  
double sqrt(double x);
```

When the user calls `sqrt` with an integer operand, are we sure that this decision was made intentionally? Is the author unaware that there is a separate function giving the integer results, or do they actually need the fractional part, and that is why they called the `double` overload? Even if the author wrote `(int) sqrt(/* ... */)`, this could plausibly be done due to performance considerations.

Similarly, calling `std::sqrt` with an integer operand could be a major performance bug on a 32-bit platform with 32-bit `float` and 64-bit `double`, considering that this is equivalent to calling `std::sqrt(double)`. Perhaps calling `std::sqrt(float)` was intended.

Conversely, if the author called `isqrt(10.f)`, the `float` $\rightarrow$ `int` conversion may be value-preserving, but this call is almost certainly a mistake. The author likely expected to obtain `3.1623f`, judging by the operand.

§ 4.7.4. Conclusion on implicit conversions

In conclusion, discrepancies between the standard integers and bit-precise integers are undesirable; they introduce a lot of unnecessary problems. There are many harmless operations like `T x = 0;` and `x & mask` where mixing signedness is okay, and not every user wants to have warnings, let alone errors for these. Especially errors would make it hard to write headers that compile both in C and in C++.

The final nail in the coffin is that if the user wants implicit conversions to be restricted, they have the freedom to add those restrictions via compiler warnings and linter checks. Having these restrictions standardized in the language robs the user of choice. If C++26 profiles make progress, it is likely that C++ will have profiles which restrict implicit conversions, giving users a standard way to opt into diagnostics.

However, the decision to permit implicit conversions is not set in stone. Especially the conversions described in §4.7.3. [Picking some low-hanging fruits](#) could be banned for bit-precise integers without much of an issue.

§ 4.8. Raising the `BITINT_MAXWIDTH`

The proposal currently does not seek to increase the `BITINT_MAXWIDTH` beyond what C offers. That is, `BITINT_MAXWIDTH` may as low as 64. I do not consider an increase of the maximum to be part of the MVP. It's something that can always be done later, if desirable, without any breaking changes.



DECISION: While the proposal does not propose an increase, some negative feedback stated that bit-precise integers as a feature are not worth the standardization effort if they only support a width of 64. Therefore, EWG should decide whether an increase should take place, and, if so, whether it should be done within this proposal.

It also should be stated that increasing the `BITINT_MAXWIDTH` is not *really* within the power of WG21 and not even within the power of compiler vendors.



EXAMPLE: Clang supports a `BITINT_MAXWIDTH` of up to 8'388'608, but only enables this for certain ABIs. For example, the `x86-64 psABI` defines an ABI for any bit-precise integer width, so the full width is available.

However, the "Basic" C ABI for WebAssembly (which Clang uses at the time of writing) has the following limitation:

”

`_BitInt(N)` types are supported up to width 128 and are represented as the smallest same-signedness Integer type with at least as many bits.

Consequently, `BITINT_MAXWIDTH` is set to 128 when compiling with `--target=wasm32-unknown-unknown`.

WG21 can define the `BITINT_MAXWIDTH` as whatever they want to; it is of no consequence because compiler vendors are not going to make that width available when there is no platform ABI for `_BitInt(BITINT_MAXWIDTH)`. If compiler vendors did that, there would be a risk of a massive future ABI break in order to comply with the system ABI, once defined. Without a single platform ABI, there would also be no portable way for code generated by different compilers to interoperate, such as compiling a C library with GCC and using it from Clang-compiled C++ code.

An increase to the `BITINT_MAXWIDTH` is political posturing. That does not mean that it's entirely pointless. If C++ defined the minimum to be, say, 65'535, this would motivate platforms to define an ABI for large bit-precise integers.

§ 4.8.1. Possible increased `BITINT_MAXWIDTH` values

Firstly, it should be noted that [P3140R0] got substantial criticism just for attempting to standardize 128-bit integers for embedded developers. As a compromise, it may be reasonable to increase the `BITINT_MAXWIDTH` only for hosted implementations, not for freestanding implementations. That being said, there two plausible increased minimums:

- 128. Many platform ABIs (see example above) already define an ABI for `_BitInt(128)`. 128-bit integers have been provided by compilers for a long time now, at least by GCC and Clang (`__int128`). There are heaps of motivation (see [P3140R0]) for 128-bit computation.

The calling conventions are also relatively obvious for 64-bit platforms: pass via pair of 64-bit integers.



- 65'535. Both GCC and Clang already support this width. Some cryptographic use cases like future-proof RSA computations need 8192 bits of key size, and at least double that for modular arithmetic. It is unlikely that a cryptographic library needs 4096 bits but does not need 8192 bits at any point, but likely that 65,535 is sufficiently large, even in the next few years.

Beyond that, `_BitInt` may be tricky to use. When working with Clang's `_BitInt(8'388'608)`, a single `+` operation could result in stack overflow because the result is 1 MiB large. The user would have to carefully ensure that all objects (including temporaries) have static or dynamic storage duration (i.e. use `new` or global variables). For these extreme sizes, a dynamically sized integer is more ergonomic. Therefore, setting the minimum to millions feels unmotivated.

§ 4.9. Template argument deduction

The following code should be valid:

```
template <std::size_t N>
void f(std::bit_int<N>);

int main() {
    f(std::bit_int<3>{}); // OK, N = 3
}
```

This would be a consequence of deduction from `_BitInt` being valid:

```
template <unsigned N>
void f(_BitInt(N));
template <int N>
void g(_BitInt(N));

int main() {
    f(_BitInt(3)(0)); // OK, N = 3
    g(_BitInt(3)(0)); // OK, N = 3
}
```

This behavior is already implemented by Clang as a C++ compiler extension, and makes deduction behave identically to deducing sizes of arrays. In general, the aim is to make the deduction of `_BitInt` widths as similar as possible to arrays because users are already familiar with the latter. It is also clearly useful because it allows writing templates that can accept `_BitInt` of any width.

While this behavior could arguably be excluded from the MVP, it would be extremely surprising to users if such deduction was not possible, given that appearance of `std::bit_int`. If deducing `N` from `std::array<T, N>` is possible, why would it not be possible to deduce `N` from `std::bit_int<N>`?

One thing deliberately not allowed is:


```
_BitInt x = 123wb;  
std::bit_int y = 123wb;
```



This class-template-argument-deduction-like construct is not part of the MVP and if desired, should be proposed separately. Even if it was allowed, `std::bit_int` is proposed to be an alias template, and alias templates do not support "forwarding deduction" to CTAD.

§ 4.10. No preprocessor changes, for better or worse

To my understanding, no changes to the preprocessor are required. [N2763] did not make any changes to the C preprocessor either. In most contexts, integer literals in the preprocessor are simply a *pp-number*, and their numeric value or type is irrelevant.

Within the controlling constant expression of an `#if` directive, all signed and unsigned integer types behave like `intmax_t` and `uintmax_t` ([cpp.cond]), which may be surprising.



EXAMPLE: The following code is ill-formed if `intmax_t` is a 64-bit signed integer (which it is on many platforms):

```
#if 1'000'000'000'000'000'000'000'000wb // error  
#endif  
_BitInt(81) x = 1'000'000'000'000'000'000'000'000wb; // OK
```

`#if 1'000'000'000'000'000'000'000'000wb` is ill-formed because the integer literal is of type `_BitInt(81)`, which behaves like `intmax_t` within `#if`. Since 10^{32} does not fit within `intmax_t`, the literal is ill-formed ([lex.icon] paragraph 4).

The current behavior could be seen as suboptimal because it makes bit-precise integers dysfunctional within the preprocessor. However, the preprocessor is largely "owned" by C, and any fix should go through WG14. In any case, fixing the C preprocessor is not part of the MVP.

§ 5. Library design

When discussing library design, it is important to understand that the vast majority of support for bit-precise integers "sneaks" into the standard without any explicit design changes or wording changes. Many existing facilities (e.g. `<bit>`) support any integer type; adding bit-precise integers to the core language silently adds library support. The following sections deal mostly with areas of the standard where some explicit design changes must be made.



NOTE: See §8.2. [Impact on the standard library](#) for a complete list of changes, including such silently added support.

§ 5.1. Naming of the alias template

The approach is to expose bit-precise integers via two alias templates:



```
template <size_t N>
    using bit_int = _BitInt(N);
template <size_t N>
    using bit_uint = unsigned _BitInt(N);
```

The goal is to have a spelling reminiscent of the C `_BitInt` spelling. There are no clear problems with it, so it is the obvious candidate. `int` and `uint` match the naming scheme of existing aliases, such as `intN_t`, `uint_fastN_t`, etc.

The alias names as a whole also act as abbreviations of the core language term (which is copied from C):

- `bit_int<N>` is a bit-precise signed integer of width `N`
- `bit_uint<N>` is a bit-precise unsigned integer of width `N`

§ 5.1.1. Why no `_t` suffix?

While the `_t` suffix would be conventional for simple type aliases such as `uint32_t`, there is no clear precedent for alias templates. There are alias templates such as `expected::rebind` without any `_t` or `_type` suffix, but "type trait wrappers" such as `conditional_t` which have a `_t` suffix.

The `_t` suffix does not add any clear benefit, adds verbosity, and distances the name from the C spelling `_BitInt`. Brevity is important here because `bit_int` is expected to be a commonly spelled type. A function doing some bit manipulation could use this name numerous times.

§ 5.2. `format`, `to_chars`, and `to_string` support for bit-precise integers

I consider printing support to be part of the MVP for bit-precise integers. There are numerous reasons for this:

- Being able to print bit-precise integers is clearly useful. It seems unthinkable that it would not be supported at some point, even if support was not added by this proposal.
- It would take considerable wording effort to exclude support for bit-precise integers from these facilities, only for it to be reverted once support is inevitably added. For example, the expression `std::println("{:b}", 100)` prints `1100100`. This is specified in terms of `std::to_chars` where `base = 2`. If `std::to_chars` does not actually support bit-precise integers, this wording becomes nonsensical.
- The design is obvious. No changes to `format` are necessary if `to_chars` supports bit-precise integers.

To facilitate printing and parsing, the following function templates are added:

```
template<class T>
    constexpr to_chars_result to_chars(char* first, char* last, T value, int base)
template<class T>
```

```
constexpr from_chars_result from_chars(char* first, char* last, T& value, int
template<class T>
    string to_string(T val);
template<class T>
    wstring to_wstring(T val);
```

NOTE: The `to_string` and `to_wstring` overloads for integral types are made `constexpr` by [P3438R0]. If that paper be accepted, the overloads for bit-precise integers should also be made `constexpr` for consistency.

NOTE: See also §5.13. [Passing `bit\_int` into standard library function templates](#) for an explanation of why this function passes by value.

`T` is constrained to accept any bit-precise integer type. It would have also been possible to accept two overloads taking `bit_int<N>` and `bit_uint<N>` with some constant template argument instead, but this doubles the amount of declarations without any clear benefit.

Such a signature is also more future-proof: the constraints can be relaxed if more types are supported (e.g. extended integer types), whereas a `bit_int<N>` parameter can only support bit-precise integer, until the end of times. For parsing and printing, this seems short-sighted.

It should also be noted that the existing overloads such as `to_string(int)` cannot be removed because it would break existing code.



EXAMPLE: Wrapper types which are convertible to `int` (but are not `int`) may rely on these dedicated overloads:

```
struct int_wrapper {
    int x;
    operator int() const { return x; }
};

string to_string(int);
string to_string_generic(integral auto);

to_string(int_wrapper{});           // OK
to_string_generic(int_wrapper{}); // error: integral<int_wrapper> constr
```

Analogously, if we replaced all the non-template overloads and handled all integers in a single function template, this may break existing valid calls to `to_string` etc.

§ 5.3. Preventing `ranges::iota_view` ABI break

Due to the current wording in [\[range.iota.view\] paragraph 1](#), adding bit-precise integers or extended integers of greater width than `long long` potentially forces the implementation to redefine `ranges::iota_view::iterator::difference_type`. Changing the type would be an ABI break. This problem is similar to historical issues with `intmax_t`, where adding 128-bit integers would force the implementation to redefine the former type.

To prevent this, the proposal tweaks the wording in § [\[range.iota.view\]](#) so that new extended or bit-precise integers may be added. Dealing with extended integer types extends slightly beyond the scope of the MVP, but it would be silly to leave the wording in an undesirable state, where adding a 128-bit extended integer still forces an ABI break.

§ 5.4. Bit-precise `size_t`, `ptrdiff_t`

As in C, the proposal allows for `size_t` and `ptrdiff_t` to be bit-precise integers, which is a consequence of `sizeof` and pointer subtraction potentially yielding a bit-precise integer.

Whether bit-precise integers in those places is desirable is for implementers and users to decide, but from the perspective of the C standard and the C++ standard, there is no compelling reason to disallow it. It would be a massive breaking change if existing C++ implementations redefined the type of these, so it is unlikely we will see an implementation that makes use of this freedom anytime soon.

§ 5.5. New `abs` overload

The proposal adds the following `abs` overload:

```
template<size_t N>
constexpr abs(bit_int<N> j);
```

While `abs` is not strictly part of the MVP, taking the absolute of an integer is such a fundamental, easy-to-implement, and useful operation that we may as well include it here.



NOTE: See §5.13. [Passing `bit\_int` into standard library function templates](#) for an explanation as to why this signature is chosen.

§ 5.6. Using bit-precise integers in `<cmath>` functions

The proposal adds support for using bit-precise integers in all `<cmath>` functions:

```
std::sqrt(0);    // OK, int → call to std::sqrt(double)
std::sqrt(0wb);  // OK, _BitInt(1) → call to std::sqrt(double)
```

This is done simply for consistency with C: after some consulting with WG14 members, I am under the impression that C's `<tgmath.h>` functions deliberately all integers types (including bit-precise integers), not just as the result of defective wording. Consequently, `_BitInt` can be passed both to the type-generic `sqrt` macro as well as to the regular `sqrt(double)` function.

§ 5.7. Lack of random number generation support



Support for random number generation is not added because too many design changes are required, with non-obvious decisions. Users can also live without bit-precise integer support in `<random>` for a while, so this feature is not part of the MVP.

Wording changes would be needed because `<random>` specifically supports certain integer types specified in `[rand.req.genl]`, rather than having blanket support for bit-precise integers.

Another issue lies with `linear_congruential_engine`. This generator performs modular arithmetic, which requires double the integer width for intermediate results. For example, `int64_t` modular arithmetic is implemented using `__int128` in some standard libraries (if available). An obvious problem for bit-precise integers is how modular arithmetic for `bit_int<BITINT_MAXWIDTH>` is meant to be implemented. We obviously can't just use a wider integer type because none exists. These and other potential design issues should be explored in a separate paper.

§ 5.8. simd support for bit-precise integers

`<simd>` is one of the few parts in the standard library where the implementation is highly specific to integer widths, at least if high implementation quality is needed.

§ 5.8.1. simd design problems

There are many important questions, such as:

- How do we best optimize `simd::vec<bit_int<1>>`? that type is effectively a bitset.
- How do we deal with padding bits? Standard integers are typically padding-free, but bit-precise integers are not except for specific sizes.
- What about `simd::vec<bit_int<512>>` or even greater widths? It seems like this case degenerates into a scalar implementation anyway.

It is not obvious whether design changes are needed to properly support bit-precise integers. Furthermore, adding a naive implementation for e.g. `bit_int<1>` would result in an ABI break when being replaced with a more efficient "bit-packed" implementation later.

§ 5.8.2. simd design conclusion

Due to these design concerns, I do not consider *fullsimd* support to be part of the MVP. However, *simd* support for bit-precise integers is clearly useful, so a compromise is possible: *simd* support is added only for those bit-precise integers whose width matches a standard integer type.

This means that a `simd::vec<bit_int<32>>` implementation "piggy-backs" off of an existing `simd::vec<int>` implementation, assuming `int` is a 32-bit signed integer. Such limited support is easy to provide.



NOTE: This restriction is inspired by the constraints (inherited from C) on `stdc_count_ones` and other `<stdbit.h>` functions. Those functions accept standard integers as well as bit-precise integers of matching width.

§ 5.9. `valarray` support for bit-precise integers

Bit-precise integer support in `valarray` is required. While the same concerns as with `<simd>` apply *in theory*, it is easy to provide a naive implementation, and the implementation in standard libraries is typically naive anyway, including for existing integers.



NOTE: Naive means that in `libc++`, `libstdc++`, and the MSVC STL, operator overloads such as `valarray::operator+` are implemented as a simple loop rather than being manually optimized with SIMD operations.

§ 5.10. Broadening `is_integral`

Since bit-precise integer types are integral types, obviously, `is_integral_v<T>` should be `true` for any bit-precise integer `T`.

There is a potential concern that existing C++ code constrained using `is_integral` or `integral` never anticipated that the templates would be instantiated with huge integers like `bit_int<1024>`. That is simply a problem we have to live with. The only way to avoid the issue would be to create a taxonomy of integer types that is confusing and inconsistent with C (e.g. by not considering bit-precise integers to be integral types), or to make `is_integral_v` inconsistent with the term "integral type". Both of these alternatives seem terrible.

§ 5.11. Miscellaneous library support

There are many more standard library parts to which support for bit-precise integers is added. Examples include:

- C headers such as `<stdbit.h>` and `<stdckdint.h>` receive the same degree of support as they have in C. This is the obvious design, and any deviation would need to be justified somehow.
- Various utilities such as `to_integer` and `hash` receive support for bit-precise integers. It is implausible that support wouldn't be added in the long run, and the support is added by extending the existing blanket support for integers; no wording changes are needed.

§ 5.12. Feature testing

After consulting with some LWG and SG10 experts, I have opted to add only two feature-test macros: one for the core feature, and one for the standard library. While more granular feature-testing could be useful considering that the feature is quite large, there seems to be little enthusiasm for it.

§ 5.13. Passing `bit_int` into standard library function templates



Unlike standard integers, it is plausible that some bit-precise integers are too large to be passed on the stack, or at least too large to make this the "default option". Nonetheless, all proposed library functions which operate on `bit_int` should accept `bit_int` by value.



EXAMPLE: The proposal adds this abs overload:

```
template<size_t N>
constexpr abs(bit_int<N> j);
```

If implemented verbatim like this, in the `x86-64 psABI`, `bit_int<64>` would be passed via single register, `bit_int<128>` would be passed via a pair of registers, and any wider integer integer would be pushed onto the stack. Passing via stack is questionable and may result in an immediate program crash when millions of bits are involved.

The reason for having such signatures is that the details of how values are passed into functions are outside the scope of the standard. Since most functions in the standard are not addressable, and since we don't care about keeping the results of reflecting on the standard library stable, the actual overload sets in the library implementation can differ from the declarations in the standard.



EXAMPLE: An implementation of the abs function template could look as follows:

```
template<size_t __n>
constexpr abs(_BitInt(__n) __j) { // pass small integers by value
    return __j >= 0 ? __j : -__j;
}

template<size_t __n>
    requires (sizeof(_BitInt(__n)) > __pass_by_value_max_size)
constexpr abs(const _BitInt(__n)& __j) { // pass large integers by reference
    return __j >= 0 ? __j : -__j;
}
```

Another plausible implementation strategy is to use an ABI-altering, implementation-specific attribute.

```
template<size_t __n>
constexpr abs([[impl::pass_large_by_ref]] _BitInt(__n) __j) {
    return __j >= 0 ? __j : -__j;
}
```

Such an attribute could alter the ABI for `__j` so that it is passed indirectly (via address) beyond a certain size, not on the stack.

Admittedly, having the standard pass all integers by value may give the user the false impression that a `bit_int<N>` function parameter (with unconstrained `N`) is idiomatic and harmless, which is problematic. However, it is seemingly the lesser evil, since the alternative is wasting LEWG and LWG time on quality of implementation.

§ 6. Education

Following SG20 Education recommendations at Sofia 2025, this proposal contains guidance on how bit-precise integers are meant to be taught by learning resources.

§ 6.1. Teaching principles

1. Emphasize familiar features. The closest equivalents to `std::bit_int` and `std::bit_uint` are `std::intN_t` and `std::uintN_t`, respectively.
2. Clearly distinguish `std::bit_int` from other existing integer types. It should be clarified that `std::bit_int` is always a distinct type from the `std::intN_t` aliases, even if it behaves similarly. Furthermore, the major differences are:
 - `std::bit_int` is not optional (though there exists a maximum width), whereas any `std::intN_t` may not actually exist.
 - `std::bit_int` is not subject to integer promotion, unlike any of the existing standard integer types.
 - `std::bit_int` cannot be used as the underlying type of enumerations.
3. Only reference the `_BitInt` spelling in a note on C compatibility. `_BitInt(N)` looks nothing like the class templates that C++ users are used to, and nothing suggests that `N` is required to be a constant expression. The `std::bit_int` and `std::bit_uint` alias templates should be taught first and foremost.
4. Point out potential pitfalls:
 - `std::bit_int` has a `BITINT_MAXWIDTH` which is not guaranteed to be any more than 64. The user should be made aware of this portability problem.
 - When writing generic code, the user should be made aware that accepting `std::bit_int<N>` in a function signature may be problematic. For all they know, `std::bit_int<N>` could have millions of bits, and this could make the type too large for passing on the stack.

§ 7. Implementation experience

`_BitInt`, formerly known as `_ExtInt`, has been a compiler extension in Clang for several years now. The core language changes are essentially standardizing that compiler extension.

§ 8. Impact on the standard



§ 8.1. Impact on the core language

The core language changes essentially boil down to adding the `_BitInt` type and the *wb integer-suffix*. This obviously comes with various syntax changes, definitions of conversion rank, addition of template argument deduction rules, etc. The vast majority of core language wording which deals with integers is not affected by the existence of bit-precise integers.

§ 8.2. Impact on the standard library

The impact of adding bit-precise integers to the standard library is quite enormous because there are many parts of the library which already support any integer type via blanket wording. Additionally, bit-precise integer support for various components such as `std::to_chars` is explicitly added.

Since this proposal does not explicitly remove support for bit-precise integers, support "sneaks" its way in, without any explicit wording changes. For example, use of bit-precise integers in `<simd>`, `<bit>`, `<valarray>`, and many others is enabled.

The addition of bit-precise integers means that (as in the core language), the `size_type` of various containers may be a bit-precise integer, `size_t` and `ptrdiff_t` may be bit-precise integers, etc.

Find a summary of affected library components below. In the interest of reducing noise, the possible changes to container `size_types` are not listed.

Header	Changes	Wording	See also
<code><algorithm></code>	Relax some <i>Mandates</i> due to implementability problems.	§ [alg.foreach]	§5.11. Miscellaneous library support
<code><atomic></code>	Add support for bit-precise integers. Partial specializations for integral types also support <code>bit_int</code> .	§ [atomics.ref.int], § [atomics.types.int]	§5.11. Miscellaneous library support
<code><bit></code>	Expand blanket support for integers.	None required	§5.11. Miscellaneous library support
<code><charconv></code>	Add <code>to_chars</code> and <code>from_chars</code> overloads.	§ [charconv.syn]	§5.2. <code>format</code> , <code>to_chars</code> , and <code>to_string</code> support for bit-precise integers
<code><chrono></code>	Bit-precise integers can be used in e.g. <code>duration</code> .	None required	
<code><climits></code>	Add <code>BITINT_MAXWIDTH</code> macro.	§ [climits.syn]	

<code><cmath></code>	Add abs overload. Allow passing bit-precise integers to most math functions.	§ [cmath.syn]	§5.5. New abs overload, §5.6. Using bit-precise integers in <cmath> functions
<code><complex></code>	Expand blanket support for integers.	None required	§5.11. Miscellaneous library support
<code><concepts></code>	Some concepts broadened (e.g. <code>integral</code>).	None required	§5.10. Broadening <code>is_integral</code>
<code><format></code>	Expand blanket support for integers.	None required	§5.2. <code>format</code>, <code>to_chars</code>, and <code>to_string</code> support for bit-precise integers
<code><limits></code>	<code>numeric_limits</code> specializations required as blanket support.	None required	
<code><limits.h></code>	Changed indirectly.	§ [climits.syn]	<code><climits></code>
<code><linalg></code>	Expand blanket support for integers.	None required	
<code><mdspan></code>	Bit-precise integers may be used as an index type.	None required	
<code><meta></code>	Some queries broadened (e.g. <code>is_integral_type</code>).	None required	§5.10. Broadening <code>is_integral</code>
<code><numeric></code>	Expand blanket support for integers (<code>gcd</code> , saturating arithmetic, etc.)	None required	
<code><ranges></code>	Change <i>IOTA-DIFF-T</i> to prevent ABI break when integer types are added.	§ [range.iota.view]	§5.3. Preventing <code>ranges::iota_view</code> ABI break
<code><simd></code>	Add limited support for bit-precise integers.	§ [simd.general]	§5.8. <code>simd</code> support for bit-precise integers
<code><stdbit.h></code>	Inherit bit-precise integer support from C.	§ [stdbit.h.syn]	§5.11. Miscellaneous library support
<code><stdckdint.h></code>	Inherit bit-precise integer support from C.	§ [numerics.c.ckdint]	§5.11. Miscellaneous library support
<code><string></code>	Add <code>to_string</code> and <code>to_wstring</code> overloads.	§ [string.conversions]	§5.2. <code>format</code>, <code>to_chars</code>, and <code>to_string</code> support for bit-precise integers
<code><tgmath.h></code>	Changed indirectly.	None required	<code><cmath></code> , <code><complex></code>
<code><type_traits></code>	Some traits broadened (e.g. <code>is_integral</code>).	None required	§5.10. Broadening <code>is_integral</code>
<code><utility></code>	Expand blanket support for integers (e.g.	None required	§5.11. Miscellaneous library support

	to_integer).		
<code><valarray></code>	Expand blanket support for integers.	None required	§5.9. valarray support for bit-precise integers 
<code><version></code>	Add feature-test macros.	§ [version.syn]	

 **NOTE:** There are numerous other standard library facilities which now support bit-precise integers, but are not mentioned specially because they are not numeric in nature. For example, it is possible to store a `bit_int` in `any`, but `<any>` is not mentioned specially in the table above.

 **NOTE:** See [\[headers\]](#) and [\[support.c.headers.general\]](#) for a complete list of headers.

§ 9. Wording

The following changes are relative to [\[N5014\]](#).

§ 9.1. Core



DECISION: CWG needs to decide what the “quoted” (prose) spelling of bit-precise integer types should be. The current spelling is e.g. “`unsigned _BitInt` of width *N*”, which is fairly similar to other code-heavy spellings like “`unsigned int`”.

However, this is questionable because `_BitInt` is not valid C++ in itself; `_BitInt(N)` is. An alternative would be a pure prose spelling, like “bit-precise unsigned integer of width *N*”, which is a bit more verbose.

There is no strong author preference.

§ [\[lex.icon\]](#)

In [\[lex.icon\]](#), change the grammar as follows:



```
integer-suffix:
    unsigned-suffix long-suffixopt
    unsigned-suffix long-long-suffixopt
    unsigned-suffix size-suffixopt
    unsigned-suffix bit-precise-int-suffixopt
    long-suffix unsigned-suffixopt
    long-long-suffix unsigned-suffixopt
    size-suffix unsigned-suffixopt
    bit-precise-int-suffix unsigned-suffixopt
```

unsigned-suffix: one of



u U

long-suffix: one of
l L

long-long-suffix: one of
ll LL

size-suffix: one of
z Z

bit-precise-int-suffix: one of
wb WB

Change table [tab:lex.icon.type] as follows:



<i>integer-suffix</i>	<i>decimal-literal</i>	<i>integer-literal</i> other than <i>decimal-literal</i>
none	"int" "long int" "long long int"	"int" "unsigned int" "long int" "unsigned long int" "long long int" "unsigned long long int"
u or U	"unsigned int" "unsigned long int" "unsigned long long int"	"unsigned int" "unsigned long int" "unsigned long long int"
l or L	"long int" "long long int"	"long int" "unsigned long int" "long long int" "unsigned long long int"
Both u or U and l or L	"unsigned long int" "unsigned long long int"	"unsigned long int" "unsigned long long int"
Both u or U and ll or LL	"unsigned long long int"	"unsigned long long int"
z or Z	the signed integer type corresponding to <u>the type named by</u> std::size_t ([support.types.layout])	the signed integer type corresponding to <u>the type named by</u> std::size_t <u>the type named by</u> std::size_t
Both u or U and z or Z	<u>the type named by</u> std::size_t	<u>the type named by</u> std::size_t
<u>wb or WB</u>	<u>"_BitInt of width N", where N is the lowest integer ≥ 1 so that the value of the literal can be represented by the type</u>	<u>"_BitInt of width N", where N is the lowest integer ≥ 1 so that the value of the literal can be represented by the type</u>
<u>Both u or U and</u>	<u>"unsigned _BitInt of width N ", where N is the lowest integer</u>	<u>"unsigned _BitInt of width N ", where N is the lowest integer</u>

<u>wb or WB</u>	<u>≥ 1 so that the value of the literal can be represented by the type</u>	<u>≥ 1 so that the value of the literal can be represented by the type</u>
-----------------	--	--



NOTE: The existing rows are adjusted for consistency. We usually aim to use the “quoted” spellings of types like “`_BitInt` of width N ” in core wording instead of the *type-id* spellings. Adding a quoted spelling for bit-precise integers would reveal that the previous rows “incorrectly” use *type-ids*.

Change [lex.icon] paragraph 4 as follows:



Except for *integer-literals* containing a *size-suffix* or *bit-precise-int-suffix*, if the value of an *integer-literal* cannot be represented by any type in its list and an extended integer type ([basic.fundamental]) can represent its value, it may have that extended integer type. [...]

[Note: An *integer-literal* with a *z* or *Z* suffix is ill-formed if it cannot be represented by `std::size_t`. An *integer-literal* with a *wb* or *WB* suffix is ill-formed if it cannot be represented by any bit-precise integer type because the necessary width is greater than `BITINT_MAXWIDTH` ([climits.syn]). — end note]

§ [basic.fundamental]

Change [basic.fundamental] paragraph 1 as follows:



There are five *standard signed integer types*: “`signed char`”, “`short int`”, “`int`”, “`long int`”, and “`long long int`”. In this list, each type provides at least as much storage as those preceding it in the list. There is also a distinct *bit-precise signed integer type* “`_BitInt` of width N ” for each $1 \leq N \leq \text{BITINT_MAXWIDTH}$ ([climits.syn]). There may also be implementation-defined *extended signed integer types*. The standard, *bit-precise*, and extended signed integer types are collectively called *signed integer types*. The range of representable values for a signed integer type is -2^{N-1} to $2^{N-1} - 1$ (inclusive), where N is called the *width* of the type.

[Note: Plain `ints` are intended to have the natural width suggested by the architecture of the execution environment; the other signed integer types are provided to meet special needs. — end note]



WARNING: This change deviates from C at the time of writing; C2y does not yet allow `_BitInt(1)`, but may allow it following [N3644].

Change [basic.fundamental] paragraph 2 as follows:



For each of the standard signed integer types, there exists a corresponding (but different) *standard unsigned integer type*: “`unsigned char`”, “`unsigned short`”, “`unsigned int`”, “`unsigned long int`”, and “`unsigned long long int`”. For each *bit-precise signed integer type* “`_BitInt` of width N ”, there exists a corresponding

bit-precise unsigned integer type “unsigned BitInt of width N ”. Likewise, for For each of the extended signed integer types, there exists a corresponding *extended unsigned integer type*. The standard, bit-precise, and extended unsigned integer types are collectively called *unsigned integer types*. An unsigned integer type has the same width N as the corresponding signed integer type. The range of representable values for the unsigned type is 0 to 2^{N-1} (inclusive); arithmetic for the unsigned type is performed modulo 2^N .

[Note: Unsigned arithmetic does not overflow. Overflow for signed arithmetic yields undefined behavior ([[expr.pre](#)]). — end note]

Change [[basic.fundamental](#)] paragraph 5 as follows:



[...] The standard signed integer types and standard unsigned integer types are collectively called the *standard integer types*, ~~and the~~ . The bit-precise signed integer types and bit-precise unsigned integer types are collectively called the bit-precise integer types. The extended signed integer types and extended unsigned integer types are collectively called the *extended integer types*.

§ [[conv.rank](#)]

Change [[conv.rank](#)] paragraph 1 as follows:



Every integer type has an *integer conversion rank* defined as follows:

- No two signed integer types other than `char` and `signed char` (if `char` is signed) have the same rank, even if they have the same representation.
- The rank of a signed integer type is greater than the rank of any signed integer type with a smaller width.
- The rank of `long long int` is greater than the rank of `long int`, which is greater than the rank of `int`, which is greater than the rank of `short int`, which is greater than the rank of `signed char`.
- The rank of any unsigned integer type equals the rank of the corresponding signed integer type.
- The rank of any standard integer type is greater than the rank of any bit-precise integer type with the same width and of any extended integer type with the same width.
- The rank of `char` equals the rank of `signed char` and `unsigned char`.
- The rank of `bool` is less than the rank of all standard integer types.
- The ranks of `char8_t`, `char16_t`, `char32_t`, and `wchar_t` equal the ranks of their underlying types ([[basic.fundamental](#)]).
- The rank of any extended signed integer type relative to another extended signed integer type with the same width and relative to a bit-precise signed integer type with the same width is implementation-defined, but still subject to the other rules for determining the integer conversion rank.

- For all integer types T1, T2, and T3, if T1 has greater rank than T2 and T2 has greater rank than T3, then T1 has greater rank than T3.



[Note: The integer conversion rank is used in the definition of the integral promotions ([conv.prom]) and the usual arithmetic conversions ([expr.arith.conv]). — end note]

§ [conv.prom]



NOTE: These changes mirror the C semantics described in [N3550] §6.3.2.1 Boolean, characters, and integers.

Change [conv.prom] paragraph 2 as follows:



A prvalue that

- is not a converted bit-field ~~and~~,
- has an integer type other than a bit-precise integer type, `bool`, `char8_t`, `char16_t`, `char32_t`, or `wchar_t`, ~~and~~
- whose integer conversion rank ([conv.rank]) is less than the rank of `int`

can be converted to a prvalue of type `int` if `int` can represent all the values of the source type; otherwise, the source prvalue can be converted to a prvalue of type `unsigned int`.

Change [conv.prom] paragraph 5 as follows:



A converted bit-field of integral type other than a bit-precise integer type can be converted to a prvalue of type `int` if `int` can represent all the values of the bit-field; otherwise, it can be converted to `unsigned int` if `unsigned int` can represent all the values of the bit-field.

§ [dcl.type.general]

Change [dcl.type.general] paragraph 2 as follows:



As a general rule, at most one *defining-type-specifier* is allowed in the complete *decl-specifier-seq* of a declaration or in a *defining-type-specifier-seq*, and at most one *type-specifier* is allowed in a *type-specifier-seq*. The only exceptions to this rule are the following:

- `const` can be combined with any type specifier except itself.
- `volatile` can be combined with any type specifier except itself.
- `signed` or `unsigned` can be combined with `char`, `long`, `short`, ~~or~~ `int`, or a bit-precise-int-type-specifier.
- `short` or `long` can be combined with `int`.
- `long` can be combined with `double`.

- `long` can be combined with `long`.



§ [dcl.type.simple]

Change [dcl.type.simple] paragraph 1 as follows:



The simple type specifiers are

```

simple-type-specifier:
    nested-name-specifieropt type-name
    nested-name-specifier template simple-template-id
    computed-type-specifier
    placeholder-type-specifier
    bit-precise-int-type-specifier
    nested-name-specifieropt template-name
    char
    char8_t
    char16_t
    char32_t
    wchar_t
    bool
    short
    int
    long
    signed
    unsigned
    float
    double
    void

type-name:
    class-name
    enum-name
    typedef-name

computed-type-specifier:
    decltype-specifier
    pack-index-specifier
    splice-type-specifier
    bit-precise-int-type-specifier:
    BitInt ( constant-expression )

```

Change table [tab:dcl.type.simple] as follows:



Specifier(s)	Type
<i>type-name</i>	the type named
<i>simple-template-id</i>	the type as defined in [temp.names]
<i>decltype-specifier</i>	the type as defined in [dcl.type.decltype]



pack-index-specifier

placeholder-type-specifier

template-name

splice-type-specifier

unsigned BitInt(*N*)

signed BitInt(*N*)

BitInt(*N*)

char

unsigned char

signed char

char8_t

char16_t

char32_t

bool

unsigned

unsigned int

signed

signed int

int

unsigned short int

unsigned short

unsigned long int

unsigned long

unsigned long long int

unsigned long long

signed long int

signed long

signed long long int

signed long long

long long int

long long

long int

long

signed short int

signed short

short int

short

wchar_t

float

double

long double

the type as defined in [dcl.type.pack.index]

the type as defined in [dcl.spec.auto]

the type as defined in [dcl.type.class.deduct]

the type as defined in [dcl.type.splice]

“unsigned BitInt of width *N*”

“ BitInt of width *N*”

“ BitInt of width *N*”

“char”

“unsigned char”

“signed char”

“char8_t”

“char16_t”

“char32_t”

“bool”

“unsigned int”

“unsigned int”

“int”

“int”

“int”

“unsigned short int”

“unsigned short int”

“unsigned long int”

“unsigned long int”

“unsigned long long int”

“unsigned long long int”

“long int”

“long int”

“long long int”

“long long int”

“long long int”

“long long int”

“long int”

“long int”

“short int”

“short int”

“short int”

“short int”

“wchar_t”

“float”

“double”

“long double”

void	"void"
------	--------



Immediately following [\[dcl.type.simple\] paragraph 3](#), add a new paragraph as follows:



Within a *bit-precise-int-type-specifier*, the *constant-expression* shall be a converted constant expression of type `std::size_t` ([\[expr.const\]](#)). Its value N specifies the width of the bit-precise integer type ([\[basic.fundamental\]](#)). The program is ill-formed unless $1 \leq N \leq \text{BITINT_MAXWIDTH}$ ([\[climits.syn\]](#)).

§ [\[dcl.enum\]](#)



NOTE: The intent is to ban `_BitInt` from being the underlying type of enumerations, matching the current restrictions in C. See [§4.3. Underlying type of enumerations](#).

Change [\[dcl.enum\] paragraph 2](#) as follows:



[...] The *type-specifier-seq* of an *enum-base* shall name an integral type other than a bit-precise integer type; any cv-qualification is ignored. [...]

Change [\[dcl.enum\] paragraph 5](#) as follows:



[...] If the underlying type is not fixed, the type of each enumerator prior to the closing brace is determined as follows:

- If an initializer is specified for an enumerator, the *constant-expression* shall be an integral constant expression ([\[expr.const\]](#)) whose type is not a bit-precise integer type. If the expression has unscoped enumeration type, the enumerator has the underlying type of that enumeration type, otherwise it has the same type as the expression.
- If no initializer is specified for the first enumerator, its type is an unspecified signed ~~integral~~ integer type other than a bit-precise integer type.
- Otherwise, the type of the enumerator is the same as that of the preceding enumerator, unless the incremented value is not representable in that type, in which case the type is an unspecified integral type other than a bit-precise integer type sufficient to contain the incremented value. If no such type exists, the program is ill-formed.

Change [\[dcl.enum\] paragraph 7](#) as follows:



For an enumeration whose underlying type is not fixed, the underlying type is an integral type that can represent all the enumerator values defined in the enumeration. If no integral type can represent all the enumerator values, the enumeration is ill-formed. It is implementation-defined which integral type is used as the underlying type, except that

- the underlying type shall not be a bit-precise integer type and

- the underlying type shall not be larger than `int` unless the value of an enumerator cannot fit in an `int` or `unsigned int`.



If the *enumerator-list* is empty, the underlying type is as if the enumeration had a single enumerator with value 0.

§ [temp.deduct.general]

Add a bullet to [temp.deduct.general] note 8 as follows:



[Note: Type deduction can fail for the following reasons:

- Attempting to instantiate a pack expansion containing multiple packs of differing lengths.
- Attempting to create an array with an element type that is `void`, a function type, or a reference type, or attempting to create an array with a size that is zero or negative.

[Example:

```
template <class T> int f(T[5]);
int I = f<int>(0);
int j = f<void>(0);           // invalid array
```

— end example]

- Attempting to create a bit-precise integer type of invalid width ([basic.fundamental]).

[Example:

```
template <int N> void f(_BitInt(N));
f<0>(0);           // invalid bit-precise integer
```

— end example]

- [...]

— end note]

§ [temp.deduct.type]

Change [temp.deduct.type] paragraph 2 as follows:



[...] The type of a type parameter is only deduced from an array bound or bit-precise integer width if it is not otherwise deduced.

Change [temp.deduct.type] paragraph 3 as follows:



A given type P can be composed from a number of other types, templates, and constant template argument values:

- A function type includes the types of each of the function parameters, the return type, and its exception specification.
- A pointer-to-member type includes the type of the class object pointed to and the type of the member pointed to.
- A type that is a specialization of a class template (e.g., `A<int>`) includes the types, templates, and constant template argument values referenced by the template argument list of the specialization.
- An array type includes the array element type and the value of the array bound.
- A bit-precise integer type includes the integer width.



Change [\[temp.deduct.type\] paragraph 5](#) as follows:



The non-deducted contexts are:

- [...]
- A constant template argument `or`, an array bound, or a bit-precise integer width, in any of which a subexpression references a template parameter.

[Example:

```
template<size_t N> void f(_BitInt(N));
template<size_t N> void g(_BitInt(N + 1));
f(100wb);                                     // OK, N = 8
g(100wb);                                     // error: no argument for ded
```

— end example]

- [...]

Change [\[temp.deduct.type\] paragraph 8](#) as follows:



A type template argument `T`, a constant template argument `i`, a template template argument `TT` denoting a class template or an alias template, or a template template argument `VV` denoting a variable template or a concept can be deduced if `P` and `A` have one of the following forms:

```
cVopt T
T*
T&
T&&
Topt[iopt]
_BitInt(iopt)
Topt(Topt) noexcept(iopt)
Topt Topt::*
TTopt<T>
TTopt<i>
TTopt<TT>
```

```
TTopt<VV>  
TTopt<>  
where [...]
```



Do not change [\[temp.deduct.type\] paragraph 14](#); it is included here for reference.

”

The type of *N* in the type *T*[*N*] is `std::size_t`.

[Example:

```
template<typename T> struct S;  
template<typename T, T n> struct S<int[n]> {  
    using Q = T;  
};  
  
using V = decltype(sizeof 0);  
using V = S<int[42]>::Q;           // OK; T was deduced as std::size_t from the typ
```

— end example]

Immediately following [\[temp.deduct.type\] paragraph 14](#), insert a new paragraph:



The type of *N* in the type `_BitInt(N)` is `std::size_t`.

[Example:

```
template <typename T, T n> void f(_BitInt(n));  
  
f(0wb);           // OK; T was deduced as std::size_t from an argu
```

— end example]

Change [\[temp.deduct.type\] paragraph 20](#) as follows:



If *P* has a form that contains `<i>`, and if the type of *i* differs from the type of the corresponding template parameter of the template named by the enclosing *simple-template-id* or *splice-specialization-specifier*, deduction fails. If *P* has a form that contains `[i] or _BitInt(i)`, and if the type of *i* is not an integral type, deduction fails. If *P* has a form that includes `noexcept(i)` and the type of *i* is not `bool`, deduction fails.

§ [cpp.predefined]

Add a feature-test macro to the table in [\[cpp.predefined\]](#) as follows:



```
__cpp_bit_int 20XXXXL
```

§ 9.2. Library



§ [version.syn]

Add the following feature-test macro to [version.syn]:

```
 #define __cpp_lib_bit_int 20XXXXL
```

§ [cstdint.syn]

In [cstdint.syn], update the header synopsis as follows:

```
 namespace std {  
    [...]  
  
    using uintmax_t = unsigned integer type;  
    using uintptr_t = unsigned integer type; // optional  
  
    template<size_t N>  
        using bit_int = _BitInt(N);  
    template<size_t N>  
        using bit_uint = unsigned _BitInt(N);  
}
```

Change [cstdint.syn] paragraph 2 as follows:

 The header defines all types and macros the same as the C standard library header <stdint.h>. None of the aliases name a bit-precise integer type. The types denoted by `intmax_t` and `uintmax_t` are not required to be able to represent all values of bit-precise integer types or of extended integer types wider than “long long int” and “unsigned long long int”, respectively.

Change [cstdint.syn] paragraph 3 as follows:

 All types that use the placeholder *N* are optional when *N* is not 8, 16, 32, or 64. The exact-width types `intN_t` and `uintN_t` for *N* = 8, 16, 32, and 64 are also optional; however, if an implementation defines integer types other than bit-precise integer types with the corresponding width and no padding bits, it defines the corresponding *typedef-names*. Each of the macros listed in this subclause is defined if and only if the implementation defines the corresponding *typedef-name*.
[Note: The macros `INTN_C` and `UINTN_C` correspond to the *typedef-names* `int_leastN_t` and `uint_leastN_t`, respectively. — end note]

§ [climits.syn]

In [climits.syn], add a new line below the definition of `ULLONG_WIDTH`:

```
 #define BITINT_MAXWIDTH see below
```


Change the synopsis in [climits.syn] paragraph 1 as follows:



The header `<climits>` defines all macros the same as the C standard library header `limits.h`, ~~except that it does not define the macro `BITINT_MAXWIDTH`.~~



§ [stdbit.h.syn]

Change [stdbit.h.syn] paragraph 2 as follows:



Mandates: `T` is ~~an unsigned integer type~~

- a standard unsigned integer type,
- an extended unsigned integer type, or
- a bit-precise unsigned integer type whose width matches a standard or extended integer type.

§ [range.iota.view]



NOTE: See §5.3. Preventing `ranges::iota_view` ABI break.

Change [range.iota.view] paragraph 1 as follows:



Let $IOTA-DIFF-T(W)$ be defined as follows:

- If `W` is not an integral type, or if it is an integral type and `sizeof(iter_difference_t<W>)` is greater than `sizeof(W)`, then $IOTA-DIFF-T(W)$ denotes `iter_difference_t<W>`.
- Otherwise, $IOTA-DIFF-T(W)$ is a standard signed integer type of width greater than the width of `W` if such a type exists.
- Otherwise, $IOTA-DIFF-T(W)$ is an unspecified signed-integer-like (`[iterator.concept.winc]`) type of width not less than the width of `W`.

§ [alg.foreach]

Change [alg.foreach] `for_each_n` as follows:



```
template<class InputIterator, class Size, class Function>
constexpr InputIterator for_each_n(InputIterator first, Size n, Function
```

Mandates: The type `Size` is convertible to an integral type other than a bit-precise integer type (`[conv.integral]`, `[class.conv]`).

[...]

```
template<class ExecutionPolicy, class ForwardIterator, class Size, class I
    ForwardIterator for_each_n(ExecutionPolicy&& exec, ForwardIterator first
        Function f);
```

Mandates: The type `Size` is convertible to an integral type other than a bit-precise integer type ([conv.integral], [class.conv]).

[...]



NOTE: Implementing this requirement for bit-precise integer types is generally impossible, barring compiler magic. The libc++ implementation is done by calling an overload in the set:

```
int __convert_to_integral(int __val) { return __val; }
unsigned __convert_to_integral(unsigned __val) { return __val; }
```

It is not reasonable to expect millions of additional overloads, and a template that can handle bit-precise integers in bulk could not interoperate with user-defined conversion function templates.

§ [alg.search]

Change [alg.search] paragraph 5 as follows:



Mandates: The type `Size` is convertible to an integral type other than a bit-precise integer type ([conv.integral], [class.conv]).

§ [alg.copy]

Change [alg.copy] paragraph 15 as follows:



Mandates: The type `Size` is convertible to an integral type other than a bit-precise integer type ([conv.integral], [class.conv]).

§ [alg.fill]

Change [alg.fill] paragraph 2 as follows:



Mandates: The expression value is writable ([iterator.requirements.general]) to the output iterator. The type `Size` is convertible to an integral type other than a bit-precise integer type ([conv.integral], [class.conv]).

§ [alg.generate]

Change [alg.generate] paragraph 2 as follows:



Mandates: Size is convertible to an integral type other than a bit-precise integer type ([conv.integral], [class.conv]).



§ [charconv.syn]

Change [charconv.syn] paragraph 1 as follows:



When a function is specified with a type placeholder of *integer-type*, the implementation provides overloads for `char` and all ~~cv-unqualified signed and unsigned integer types~~ standard and extended integer types in lieu of *integer-type*. When a function is specified with a type placeholder of *floating-point-type*, the implementation provides overloads for all cv-unqualified floating-point types ([basic.fundamental]) in lieu of *floating-point-type*.

```
namespace std {
    // floating-point format for primitive numerical conversion
    enum class chars_format {
        scientific = unspecified,
        fixed = unspecified,
        hex = unspecified,
        general = fixed | scientific
    };

    // [charconv.to.chars], primitive numerical output conversion
    struct to_chars_result {
        char* ptr;
        errc ec;
        friend bool operator==(const to_chars_result&, const to_chars_result&);
        constexpr explicit operator bool() const noexcept { return ec == errc::no_error; }
    };

    constexpr to_chars_result to_chars(char* first, char* last,
                                       integer-type value, int base = 10);

    template<class T>
        constexpr to_chars_result to_chars(char* first, char* last,
                                           T value, int base = 10);

    to_chars_result to_chars(char* first, char* last,
                            bool value, int base = 10) = delete;

    to_chars_result to_chars(char* first, char* last,
                            floating-point-type value);
    to_chars_result to_chars(char* first, char* last,
                            floating-point-type value, chars_format fmt);
    to_chars_result to_chars(char* first, char* last,
                            floating-point-type value, chars_format fmt, ir

    // [charconv.from.chars], primitive numerical input conversion
    struct from_chars_result {
        const char* ptr;
        errc ec;
        friend bool operator==(const from_chars_result&, const from_chars_resu
```



```

constexpr explicit operator bool() const noexcept { return ec == error_code::no_error; }

constexpr from_chars_result from_chars(const char* first, const char* last,
                                       integer-type& value, int base = 10);

template<class T>
constexpr from_chars_result from_chars(char* first, char* last,
                                       T& value, int base = 10);

from_chars_result from_chars(const char* first, const char* last,    //
                             floating-point-type& value,          //
                             chars_format fmt = chars_format::general);

```

§ [charconv.to.chars]

Change [charconv.to.chars] as follows:



[...]

```

constexpr to_chars_result to_chars(char* first, char* last, integer-type& value,
template<class T>
constexpr to_chars_result to_chars(char* first, char* last, T value, int

```

Constraints: T is a bit-precise integer type.

Preconditions: base has a value between 2 and 36 (inclusive).

[...]

§ [charconv.from.chars]

Change [charconv.from.chars] as follows:



[...]

```

constexpr from_chars_result from_chars(const char* first, const char* last,
                                       integer-type& value, int base = 10);

template<class T>
constexpr from_chars_result from_chars(const char* first, const char* last,
                                       T& value, int base = 10);

```

Constraints: T is a bit-precise integer type.

Preconditions: base has a value between 2 and 36 (inclusive).

[...]



Change [string.syn] as follows:



```
namespace std {  
    [...]  
  
    string to_string(int val);  
    string to_string(unsigned val);  
    string to_string(long val);  
    string to_string(unsigned long val);  
    string to_string(long long val);  
    string to_string(unsigned long long val);  
    string to_string(float val);  
    string to_string(double val);  
    string to_string(long double val);  
    template<class T> string to_string(T val);  
  
    [...]  
  
    wstring to_wstring(int val);  
    wstring to_wstring(unsigned val);  
    wstring to_wstring(long val);  
    wstring to_wstring(unsigned long val);  
    wstring to_wstring(long long val);  
    wstring to_wstring(unsigned long long val);  
    wstring to_wstring(float val);  
    wstring to_wstring(double val);  
    wstring to_wstring(long double val);  
    template<class T> wstring to_wstring(T val);  
  
    [...]  
}
```

If the existing overloads for integral types have been made `constexpr` through [P3438R0] or a subsequent paper, additionally make the following changes:



```
[...]  
template<class T> constexpr string to_string(T val);  
[...]  
template<class T> constexpr wstring to_wstring(T val);  
[...]
```

Change [string.conversions] as follows:



```
[...]  
  
string to_string(int val);  
string to_string(unsigned val);  
string to_string(long val);
```



```
string to_string(unsigned long val);
string to_string(long long val);
string to_string(unsigned long long val);
string to_string(float val);
string to_string(double val);
string to_string(long double val);
template<class T> string to_string(T val);
```

Constraints: T is a bit-precise or extended integer type.

Returns: `format("{} ", val)`.

[...]

```
wstring to_wstring(int val);
wstring to_wstring(unsigned val);
wstring to_wstring(long val);
wstring to_wstring(unsigned long val);
wstring to_wstring(long long val);
wstring to_wstring(unsigned long long val);
wstring to_wstring(float val);
wstring to_wstring(double val);
wstring to_wstring(long double val);
template<class T> wstring to_wstring(T val);
```

Constraints: T is a bit-precise or extended integer type.

Returns: `format(L"{} ", val)`.

[...]

If the existing overloads for integral types have been made `constexpr` through [P3438R0] or a subsequent paper, additionally make the following changes:



```
[...]
template<class T> constexpr string to_string(T val);
[...]
template<class T> constexpr wstring to_wstring(T val);
[...]
```

§ [cmath.syn]



NOTE: [cmath.syn] paragraph 3 is deliberately not changed, meaning that `bit_int` may be passed to e.g. `sqrt`. See §5.6. Using bit-precise integers in `<cmath>` functions.

In [cmath.syn], change the synopsis as follows:



```
constexpr int abs(int j); // freest
constexpr long int abs(long int j); // freest
constexpr long long int abs(long long int j); // freest
```

```
template<size_t N> constexpr abs(bit_int<N> j); // freest
constexpr floating-point-type abs(floating-point-type j); // freest
```

§ [c.math.abs]

NOTE: See §5.5. New abs overload.

Change [c.math.abs] as follows:

```
constexpr int abs(int j);
constexpr long int abs(long int j);
constexpr long long int abs(long long int j);
template<size_t N> constexpr abs(bit_int<N> j);
```

~~Effects: These functions have the semantics specified in the C standard library for the functions abs, labs, and llabs, respectively.~~

~~Remarks: If abs is called with an argument of type X for which is_unsigned_v<X> is true and if X cannot be converted to int by integral promotion, the program is ill-formed.~~

~~[Note: Allowing arguments that can be promoted to int provides compatibility with C. —end note]~~

Effects: Equivalent to $j \geq 0 ? j : -j$.

[Note: The behavior is undefined if j has the lowest possible integer value of its type ([expr.pre]). — end note]

NOTE: Specifying the undefined behavior as a *Preconditions* specification would be worse because it may cause library UB during constant evaluation.

The *Effects* specification needs to be altered because abs for bit-precise integers is a novel invention with no C counterpart. It also seems like unnecessary indirection to refer to another language standard for a single expression.

The *Remarks* specification is removed because it is a usage tutorial and history lesson; it does not say anything about what abs does. The specification is also factually wrong. Just because an attempt is made to call abs(0u) and the overloads above don't handle it, doesn't mean that the user doesn't have their own abs(unsigned) overload. In that event, the program is not ill-formed; overload resolution simply doesn't select one of these functions.

§ [simd.general]

Change [simd.general] paragraph 2 as follows:

The set of *vectorizable types* comprises

- all standard integer types, character types, and the types `float` and `double` ([[basic.fundamental](#)]);
- any bit-precise integer type whose width matches a standard integer type;
- `std::float16_t`, `std::float32_t`, and `std::float64_t` if defined ([[basic.extended.fp](#)]); and
- `complex<T>` where T is a vectorizable floating-point type.



§ [numerics.c.ckdint]

Change [[numerics.c.ckdint](#)] as follows:



```
template<class type1, class type2, class type3>
    bool ckd_add(type1* result, type2 a, type3 b);
template<class type1, class type2, class type3>
    bool ckd_sub(type1* result, type2 a, type3 b);
template<class type1, class type2, class type3>
    bool ckd_mul(type1* result, type2 a, type3 b);
```

Mandates: type1 is a signed or unsigned integer type. Each of the types `type1`, `type2`, and `type3` is a cv-unqualified signed or unsigned integer type other than a bit-precise integer type.

Remarks: Each function template has the same semantics as the corresponding type-generic macro with the same name specified in ISO/IEC 9899:2024, 7.20.



NOTE: This matches the restrictions in [[N3550](#)], 7.20 "Checked Integer Arithmetic". "cv-unqualified" is struck because it is redundant.

§ [atomics.ref.int]

Do not change [[atomics.ref.int](#)] paragraph 1; it is provided here for reference:



There are specializations of the `atomic_ref` class template for all integral types except cv `bool`. For each such type *integral-type*, the specialization `atomic_ref<integral-type>` provides additional atomic operations appropriate to integral types.

§ [atomics.types.int]

Change [[atomics.types.int](#)] paragraph 1 as follows:



There are specializations of the `atomic` class template for ~~the integral types `char`, `signed char`, `unsigned char`, `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `char8_t`, `char16_t`, `char32_t`, `wchar_t`~~; standard integer types, bit-precise integer types, character types, and any other types needed by the typedefs in the header `<cstdint>` ([[cstdint.syn](#)]). For each

such type *integral-type*, the specialization `atomic<integral-type>` provides additional atomic operations appropriate to integral types.



[Note: The specialization `atomic<bool>` uses the primary template (`[atomics.types.generic]`). — *end note*]

§ 10. Acknowledgements

I thank Jens Maurer and Christof Meerwald for reviewing and correcting the proposal's wording.

I thank Erich Keane and other LLVM contributors for implementing most of the proposed core changes in Clang's C++ frontend, giving this paper years worth of implementation experience in a major compiler without any effort by the author.

I thank Erich Keane, Bill Seymour, Howard Hinnant, JeanHeyd Meneide, Lénárd Szolnoki, Brian Bi, Peter Dimov, Aaron Ballman, Pete Becker, Jens Maurer, Matthias Kretz, Jonathan Wakely, Jeff Garland, Ville Voutilainen, Peter Dimov, Luigi Ghiron, and *many* others for providing early feedback on this paper, prior papers such as [P3639R0], and the discussion surrounding bit-precise integers as a whole. The paper would not be where it is today without *hundreds* of messages worth of valuable feedback.

§ 11. References

[N1692] *M.J. Kronenburg*. A Proposal to add the Infinite Precision Integer to the C++ Standard Library (2004-07-01)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2004/n1692.pdf>

[N1744] *Michiel Salters*. Big Integer Library Proposal for C++0x (2005-01-13)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2005/n1744.pdf>

[N4038] *Pete Becker*. Proposal for Unbounded-Precision Integer Types (2014-05-23)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4038.html>

[N5014] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-08-05)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf>

[P3140R0] *Jan Schultke*. `std::int_least128_t` (2025-02-11)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3140r0.html>

[P3161R4] *Tiago Freire*. Unified integer overflow arithmetic (2025-03-24)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3161r4.html>

[P3639R0] *Jan Schultke*. The `_BitInt` Debate (2025-02-20)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3639r0.html>

[P3312R1] *Bengt Gustafsson*. Overload Set Types (2025-04-16)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3312r1.pdf>

[P3438R0] *Andreas Fertig*. Make integral overloads of `std::to_string constexpr` (2024-10-13)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3438r0.pdf>



[N2763] *Aaron Ballman, Melanie Blower, Tommy Hoffner, Erich Keane*. Adding a Fundamental Type for N-bit integers (2021-06-21)

<https://open-std.org/JTC1/SC22/WG14/www/docs/n2763.pdf>

[N2775] *Aaron Ballman, Melanie Blower*. Literal suffixes for bit-precise integers (2021-07-13)

<https://open-std.org/JTC1/SC22/WG14/www/docs/n2775.pdf>

[N3550] *JeanHeyd Meneide*. ISO/IEC 9899:202y (en) — N3550 working draft (2025-05-04)

<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3550.pdf>

[N3644] *Robert C. Seacord*. Integer Sets, v2 (2025-07-05)

<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3644.pdf>